

实 验 报 告

课程名称：机器学习及优化

姓名：王志豪

学号：08232337

班级：2023 大数据一班

日期：2026. 4. 12

中国矿业大学计算机学院/人工智能学院

实验名称 聚类分析 实验日期 2026. 4. 12

实验报告内容: 1. 实验目的 2. 实验内容 3. 实验步骤 4. 实验结果 5. 实验体会

一、实验目的

通过实验更好地掌握机器学习及优化相关概念、技术、原理、应用等，提高编写实验报告、总结实验结果的能力；对机器学习模型、优化算法等有比较深入的认识。需掌握的知识点如下：

- 掌握机器学习中涉及的相关概念、模型、算法；
- 熟悉机器学习模型训练、验证、测试的流程；
- 熟悉常用的数据预处理方法；
- 掌握聚类分析问题的表示、求解及编程。

二、实验内容

基于 IRIS 鸢尾花数据集，完成关于鸢尾花的聚类分析。

(1) 准备数据集并认识数据

下载 IRIS 数据集

<https://archive.ics.uci.edu/ml/datasets/iris>

了解数据集各个维度特征的含义

(2) 探索数据并预处理数据

观察数据集各个维度特征的数值类型与分布

挑选 `sepalength`、`petallength` 两维特征作为聚类依据

(3) 求解聚类中心

编程实现 `k-means` 聚类、混合高斯聚类

(4) 测试和评估模型

在数据集上计算聚类的性能指标

三、实验步骤

实验原理：

K-means 聚类算法：

`k-means` 是一种常用的聚类算法。它的输入是一个样本集合，输出是将样本划分为若干个类别，使得同一类别内的样本具有较高的相似性。

假设我们希望将数据分成 K 类，算法流程如下：

- (1). 随机选择 K 个样本点作为初始的聚类中心。
- (2). 计算每个样本点到各个聚类中心的距离，并将其划分到距离最近的中心所对应的类别中，形成 K 个簇。
- (3). 对每个簇，重新计算其质心（即簇内所有样本的均值）。
- (4). 重复第 2 和第 3 步，直到聚类中心不再发生变化或达到预设的迭代次数。

混合高斯聚类原理：

1. 多元高斯分布的概率密度函数

$$p(x) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} e^{-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)}$$

其中：

- x 是一个 d 维样本向量；
- μ 是 d 维均值向量；
- Σ 是 $d \times d$ 的协方差矩阵（对称半正定）；
- $|\Sigma|$ 是协方差矩阵的行列式；
- Σ^{-1} 是其逆矩阵。

该函数返回样本 x 在 高斯分布下的概率密度值。可以看出，概率密度由均值 μ 和协方差矩阵 Σ 决定。

2. 高斯混合模型的基本思想

高斯混合模型假设观测数据是从若干个高斯分布中混合生成的。其概率密度函数为：

$$P(x | \theta) = \sum_{k=1}^K a_k p(x | \mu_k, \Sigma_k)$$

其中 $\theta = (a_1, \dots, a_K; \mu_1, \dots, \mu_K; \Sigma_1, \dots, \Sigma_K)$ 为模型参数， a_k 是第 k 个高斯分量的权重，满足 $\sum a_k = 1$ 。

3. 参数估计方法：EM 算法

我们使用期望最大化（EM）算法来估计高斯混合模型的参数。

（1）初始化参数

为每个高斯分量 $k = 1, \dots, K$ 初始化均值 $\hat{\mu}_k$ 、协方差矩阵 $\hat{\Sigma}_k$ 和权重 $\hat{a}_k$ 。

（2）E 步（期望步）

计算每个样本 x_j 属于第 k 个高斯分量的后验概率：

$$\hat{y}_{jk} = \frac{a_k p(x_j | \mu_k, \Sigma_k)}{\sum_{i=1}^K a_i p(x_j | \mu_i, \Sigma_i)}, j = 1, \dots, N, k = 1, \dots, K$$

该值反映了第 k 个高斯分量对样本 x_j 的“责任”或“影响程度”，类似于朴素贝叶斯中的后验概率计算。

（3）M 步（最大化步）

基于 E 步计算出的责任值，更新模型参数：

- 更新均值：

$$\hat{\mu}_k = \frac{\sum_{j=1}^N \hat{y}_{jk} x_j}{\sum_{j=1}^N \hat{y}_{jk}}$$

- 更新协方差矩阵：

$$\hat{\Sigma}_k = \frac{\sum_{j=1}^N \hat{y}_{jk} (x_j - \hat{\mu}_k)(x_j - \hat{\mu}_k)^T}{\sum_{j=1}^N \hat{y}_{jk}}$$

- 更新权重：

$$\hat{a}_k = \frac{\sum_{j=1}^N \hat{y}_{jk}}{N}$$

4. 迭代收敛

重复执行 E 步和 M 步，直到模型参数趋于稳定或达到设定的收敛条件。

K-means 聚类算法：

1. 导入相应库

```
import numpy as np
import pandas as pd
import random
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
```

图 1. 导入库

2. 加载数据集，观察数据特点

```
iris_data=pd.read_csv("Iris/iris.data", header=None, names=['sepal length', 'sepal width', 'petal length', 'petal width', 'class'])
print(iris_data.info())

#发现有 iris-setosa iris-versicolor iris-virginica 三种
print(iris_data['class'].value_counts())
```

图 2. 加载数据集

```
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   sepal length    150 non-null    float64
1   sepal width     150 non-null    float64
2   petal length    150 non-null    float64
3   petal width     150 non-null    float64
4   class           150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
None
class
Iris-setosa      50
Iris-versicolor  50
Iris-virginica   50
Name: count, dtype: int64
```

图 3. 数据集描述

3. 挑选 sepal length、petal length 两维特征作为聚类依据，将类别“class”列的值赋给 labels，并进行标签编码


```

#--开始训练 训练 100 次--
for i in range(1000):
    # 用来装分属于三类的数据的 index 值
    belong1 = []
    belong2 = []
    belong3 = []

    # 计算每条分类数据到 3 个聚类中心的距离
    for j in range(150):
        belong=0 #belong 用来记录该条数据属于的类别
        dis_1=pow((x_axis[j]-x_center1),2)+pow((y_axis[j]-y_center1),2)
        dis_2=pow((x_axis[j]-x_center2),2)+pow((y_axis[j]-y_center2),2)
        dis_3=pow((x_axis[j]-x_center3),2)+pow((y_axis[j]-y_center3),2)

        # 比较离三类中心点哪个更近, 将该条数据归于距离更近的中心点所属类
        if dis_2<dis_1:
            belong=2
        if dis_3<dis_2:
            belong=3
        else:
            belong=1
        if dis_3<dis_1:
            belong=3

        print(belong)
        if belong==1:
            belong1.append(j)
        elif belong==2:
            belong2.append(j)
        else:
            belong3.append(j)

```

图 8. 原训练代码

实际上参考的代码中有一项逻辑的错误, 就是在比较距离的时候, 无法正确表示"选择距离最小的中心":

```

if dis_2<dis_1:
    belong=2
if dis_3<dis_2:
    belong=3
else:
    belong=1
if dis_3<dis_1:
    belong=3

```

第 3 个 if 语句的 else 分支会无条件地将 belong 设为 1, 这会覆盖之前的判断。更改后的结果如下:

```

if dis_1 <= dis_2 and dis_1 <= dis_3:
    belong=1
elif dis_2 <= dis_1 and dis_2 <= dis_3:
    belong=2
else:
    belong=3

```

图 9. 修正代码

6. 重新计算每个簇的质心 (均值)

```

# 进行 center 点的位置更新
for k in range(len(belong1)):
    x_center1+=x_axis[belong1[k]]
    y_center1+=y_axis[belong1[k]]

for k in range(len(belong2)):
    x_center2+=x_axis[belong2[k]]
    y_center2+=y_axis[belong2[k]]

for k in range(len(belong3)):
    x_center3+=x_axis[belong3[k]]
    y_center3+=y_axis[belong3[k]]

x_center1=x_center1/(1+len(belong1))
x_center2=x_center2/(1+len(belong2))
x_center3=x_center3/(1+len(belong3))

y_center1=y_center1 / (1 + len(belong1))
y_center2=y_center2 / (1 + len(belong2))
y_center3=y_center3 / (1 + len(belong3))

```

图 10. 重新计算每个簇的质心

7. 迭代 5, 6 步 max_iter = 1000 次
8. 显示 k-means 算法的聚类情况, 同时也显示实际数据的归类情况

```

y_pred=np.array(np.zeros(150))

for i in range(len(belong1)):
    y_pred[belong1[i]]=1

for i in range(len(belong2)):
    y_pred[belong2[i]]=2

for i in range(len(belong3)):
    y_pred[belong3[i]]=3

#k-means 求出的聚类中心
x_center=[x_center1, x_center2, x_center3]
y_center=[y_center1, y_center2, y_center3]

# 数据集的实际中心
x_ac_center=[x_axis[0:50].mean(), x_axis[50:100].mean(), x_axis[100:150].mean()]
y_ac_center=[y_axis[0:50].mean(), y_axis[50:100].mean(), y_axis[100:150].mean()]

# 聚类算法的归类情况
plt.scatter(x_axis,y_axis,c=y_pred)
plt.scatter(x_center,y_center,c='r',marker='x')
plt.show()

# 实际数据的归类情况
plt.scatter(x_axis,y_axis,c=labels)
plt.scatter(x_ac_center,y_ac_center,c='r',marker='x')
plt.show()

```

图 11. 显示归类结果

使用 k_means 聚类算法的聚类结果如下:

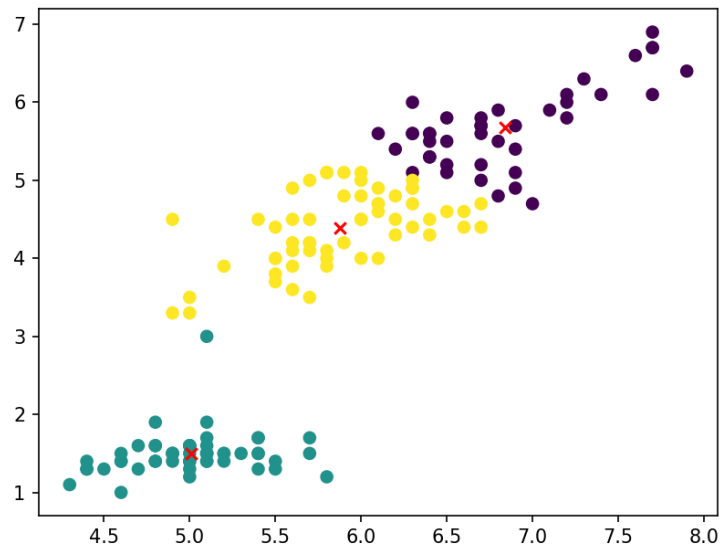


图 12. k_means 聚类算法的聚类结果

真实分类结果:

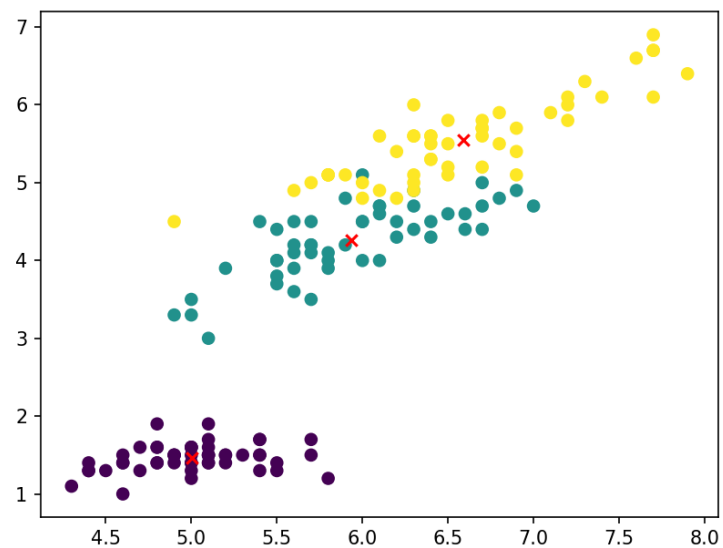


图 13. 真实分类结果

9. 计算准确率

```
# 计算准确率
# 分别计算三种类别组合 0 1 2 1 0 2 0 2 1 1 2 0 2 1 0 2 0 1
y_pred_1=np.array(np.zeros(150))
y_pred_2=np.array(np.zeros(150))
y_pred_3=np.array(np.zeros(150))
y_pred_4=np.array(np.zeros(150))
y_pred_5=np.array(np.zeros(150))
y_pred_6=np.array(np.zeros(150))

for i in range(150):
    if y_pred[i]==1:
        y_pred_1[i]=0
```



```

        y_pred_2[i]=1
        y_pred_3[i]=0
        y_pred_4[i]=1
        y_pred_5[i]=2
        y_pred_6[i]=2
    if y_pred[i]==2:
        y_pred_1[i]=1
        y_pred_2[i]=0
        y_pred_3[i]=2
        y_pred_4[i]=2
        y_pred_5[i]=1
        y_pred_6[i]=0
    if y_pred[i]==3:
        y_pred_1[i]=2
        y_pred_2[i]=2
        y_pred_3[i]=1
        y_pred_4[i]=0
        y_pred_5[i]=0
        y_pred_6[i]=1

def correct_rate(lei_list):
    correct_num = 0
    for i in range(150):
        if (lei_list[i] == labels[i]):
            correct_num += 1
    rate = correct_num / 150
    return rate

rate1=correct_rate(y_pred_1)
rate2=correct_rate(y_pred_2)
rate3=correct_rate(y_pred_3)
rate4=correct_rate(y_pred_4)
rate5=correct_rate(y_pred_5)
rate6=correct_rate(y_pred_6)

#比较
rate=[rate1, rate2, rate3, rate4, rate5, rate6]
max_rate=0
for i in range(6):
    if rate[i]>max_rate:
        max_rate=rate[i]

print('准确率为: ',max_rate)

```

由于聚类算法只会将原始数据样本划分为 K 个簇，但是并不会告诉我们每个簇 分别对应哪个类别，因此

用排列组合的方法，分别计算每种情况的准确率，选最高的为最终的准确率值：

准确率为： 0.88

图 14. Kmeans 聚类准确率

混合高斯方法：

1. 引入相应库

```
from scipy.stats import multivariate_normal
from sklearn import preprocessing
from sklearn.metrics import accuracy_score
import numpy as np
import pandas as pd
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt
```

图 15. 导入库

2. 实例化类 GMM_EM 的对象 gmm

关于 GMM_EM 中的方法报告：

方法名	功能	输入输出
<code>__init__(n_components, max_iter, error)</code>	初始化算法参数	设置聚类簇数和迭代条件
<code>preprocess(data)</code>	数据预处理	特征归一化到[0,1]范围，保存样本数和特征数
<code>_init(data)</code>	模型参数初始化	随机初始化 μ 、 σ 、 α 参数
<code>gauss(Y, mu, sigma)</code>	计算高斯概率密度	调用 <code>scipy</code> 多元正态分布 pdf 函数
<code>_e_step(data)</code>	E 步（期望步）	计算每个样本属于各分模型的后验概率
<code>_m_step(data, weighted_probs)</code>	M 步（最大化步）	基于后验概率更新 μ 、 σ 、 α 参数
<code>fit_predict(data)</code>	主训练过程	调用 E 步和 M 步交替迭代，返回聚类标签

表 1. GMM_EM 中的方法报告

实例化 GMM_EM 类：

```
gmm = GMM_EM(3)
```

图 15. 实例化 GMM_EM

3. 调用对象 gmm 中的 fit_predict 函数，得到用混合高斯模型进行聚类的结果

```

def fit_predict(self, data): # 拟合数据
    data = self.preprocess(data) # 进行数据预处理
    self._init(data) # 初始化模型参数
    weighted_probs = np.zeros((self.samples, self.n_components))
    print(weighted_probs.shape) # 用来存放进行 E 步后算出来的当前模型参数下每个观测数据
    # 来自第 k 个分模型的概率 shape(150,3)
    for i in range(self.max_iter):
        prev_weighted_probs = weighted_probs
        # E 步
        weighted_probs = self._e_step(data)
        # change 当后验概率没有什么改变了的时候, 即收敛的时候, 停止迭代
        change = np.linalg.norm(weighted_probs - prev_weighted_probs)
        if change < self.error:
            break
        # M 步
        self._m_step(data, weighted_probs)
    # 比较观测数据来自三个分模型的概率, 返回概率最大的分模型所代表列的列号
    return weighted_probs.argmax(axis=1)

```

图 16. 调用对象 gmm 中的 fit_predict 函数

关于 fit_predict 的函数内步骤:

- 1) 进行数据预处理, 调用类内函数 preprocess()

```

def preprocess(self, data): # 数据预处理
    self.samples = data.shape[0] # 定义数据集大小
    self.features = data.shape[1] # 定义数据集的特征数
    pre = preprocessing.MinMaxScaler() # 进行了特征归一化
    return pre.fit_transform(data)

```

图 17. Preprocess 函数定义

data 数据集中大小为 150, 特征数为 2。

- 2) 调用类内函数 _init() 初始化高斯模型的参数

```

def _init(self, data): # 初始化参数
    np.random.seed(4)
    self.mu = np.array(np.random.rand(self.n_components, self.features))
    # sigma 中为每个分模型都初始化了一个二维随机变量的协方差矩阵
    self.sigma = np.array([np.eye(self.features) / self.features] * self.n_components)
    self.alpha = np.array([1.0 / self.n_components] * self.n_components)
    print(self.alpha.shape, self.mu.shape, self.sigma.shape)
    print(self.alpha, self.mu, self.sigma)

```

图 18. _init 函数

- 3) 在 max_iter 迭代次数为 1000 的情况下执行 EM 算法中的 E、M 步骤, 当前后两次迭代概率 的变化 $< 1e-6$ 时, 即可跳出迭代。

E 步骤算法原理:

$$\hat{y}_{jk} = \frac{\alpha_k \phi(y_j | \theta_k)}{\sum_{k=1}^K \alpha_k \phi(y_j | \theta_k)}, j=1, 2, \dots, N, k=1, 2, \dots, K$$

```
def _e_step(self, data): # E 步
    probs = np.zeros((self.samples, self.n_components)) #shape(150,3)
    for i in range(self.n_components):
        #调用类定义的 gauss 函数，计算不同高斯模型下数据集对应高斯函数值
        probs[:, i] = self.gauss(data, self.mu[i, :], self.sigma[i, :])

    weighted_probs = np.zeros((probs.shape))
    for i in range(self.n_components):
        weighted_probs[:, i] = self.alpha[i] * probs[:, i]

    for i in range(self.samples):
        # 后验概率某类概率再除以三类的概率和
        weighted_probs[i, :] /= np.sum(weighted_probs[i, :])

    return weighted_probs
```

图 19. _e_step 函数定义

gauss()函数定义如下:

```
def gauss(self, Y, mu, sigma): #调用多元正态分布的概率密度函数，计算高斯函数值
    return multivariate_normal.pdf(Y, mean=mu, cov=sigma)
```

图 20. Gauss 函数定义

M 步骤的函数对应算法:

$$\hat{\mu}_k = \frac{\sum_{j=1}^N \hat{\gamma}_{jk} \mathbf{x}_j}{\sum_{j=1}^N \hat{\gamma}_{jk}}$$

$$\hat{\alpha}_k = \frac{\sum_{j=1}^N \hat{\gamma}_{jk}}{N}$$

```
def _m_step(self, data, weighted_probs): # M 步 进行 mu,sigma,alpha 的数值更新
    for i in range(self.n_components):
        # 求出每一列的概率和 即每行数据属于某一具体类的概率和
        sum_probs_i = np.sum(weighted_probs[:, i])
        #axis=0 计算每列的 sum
        self.mu[i, :] = np.sum(np.multiply(data, np.asmatrix(weighted_probs[:, i]).T), axis=0) / sum_probs_i

        self.sigma[i, :, :] = (data - self.mu[i, :]).T * np.multiply((data - self.mu[i, :]), np.asmatrix(weighted_probs[:, i]).T) / sum_probs_i

    #行数 shape[0]
    self.alpha[i] = sum_probs_i / data.shape[0]
```

图 21. _m_step 函数定义

4. 求解混合高斯模型聚类中心

```

# 混合高斯算法求出的聚类中心
num_0, num_1, num_2=0,0,0
xsum_0, xsum_1, xsum_2=0,0,0
ysum_0, ysum_1, ysum_2=0,0,0

for i in range(len(pre_label)):
    if pre_label[i]==0:
        num_0+=1
        xsum_0+=x_axis[i]
        ysum_0+=y_axis[i]
    elif pre_label[i]==1:
        num_1+=1
        xsum_1+=x_axis[i]
        ysum_1+=y_axis[i]
    else:
        num_2+=1
        xsum_2+=x_axis[i]
        ysum_2+=y_axis[i]

x_center_0=xsum_0/num_0
y_center_0=ysum_0/num_0
x_center_1=xsum_1/num_1
y_center_1=ysum_1/num_1
x_center_2=xsum_2/num_2
y_center_2=ysum_2/num_2

x_center=[x_center_0, x_center_1, x_center_2]
y_center=[y_center_0, y_center_1, y_center_2]

# 数据集的实例中心
x_ac_center=[x_axis[0:50].mean(), x_axis[50:100].mean(), x_axis[100:150].mean()]
y_ac_center=[y_axis[0:50].mean(), y_axis[50:100].mean(), y_axis[100:150].mean()]

```

图 22. 求解混合高斯模型聚类中心

5. 画图显示混合高斯聚类结果的聚类情况及中心点，同时也显示实际数据的归类情况

```

# 画图
## 画混合高斯聚类图
plt.scatter(x_axis, y_axis, c=pre_label)
plt.scatter(x_center, y_center, c='r', marker='x')
plt.xlabel('sepal length')
plt.ylabel('petal length')
plt.show()

# 实际数据图
plt.scatter(x_axis, y_axis, c=labels)
plt.scatter(x_ac_center, y_ac_center, c='r', marker='x')
plt.xlabel('sepal length')
plt.ylabel('petal length')
plt.show()

```

图 23. 聚类情况

混合高斯聚类结果:

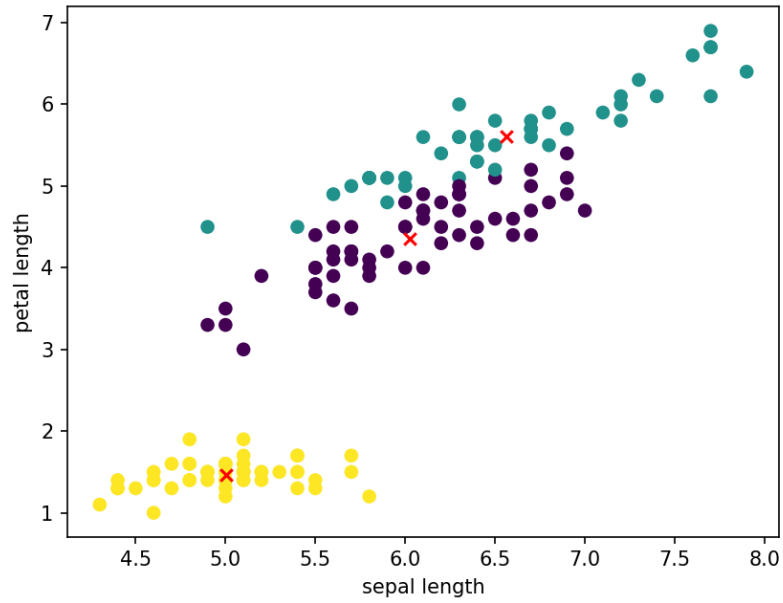


图 24. 混合高斯聚类结果

真实分类结果:

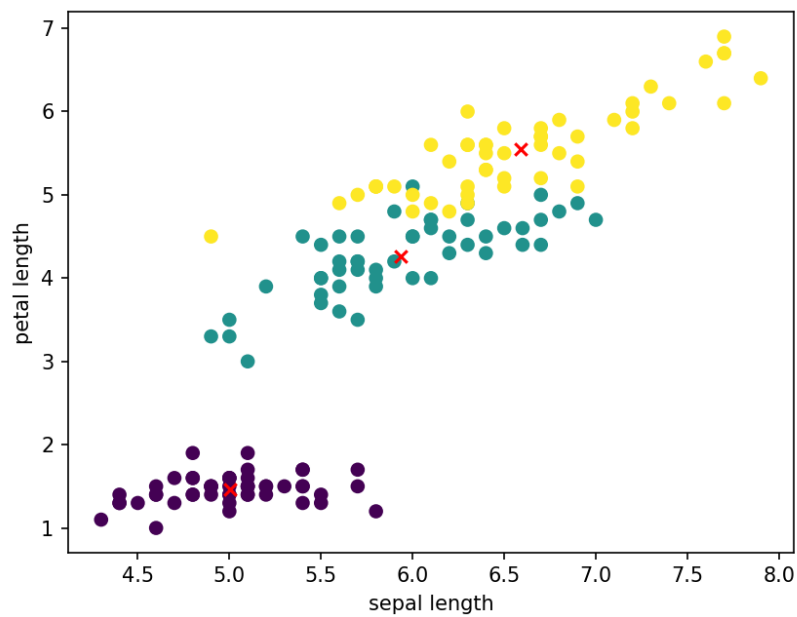


图 25. 真实分类结果

6. 计算准确率，方法与 K-means 中的相同

准确率为: 0.92

图 26. 混合高斯聚类准确率

四、实验结果

K-means 聚类算法:

1. k-means 算法的聚类情况:

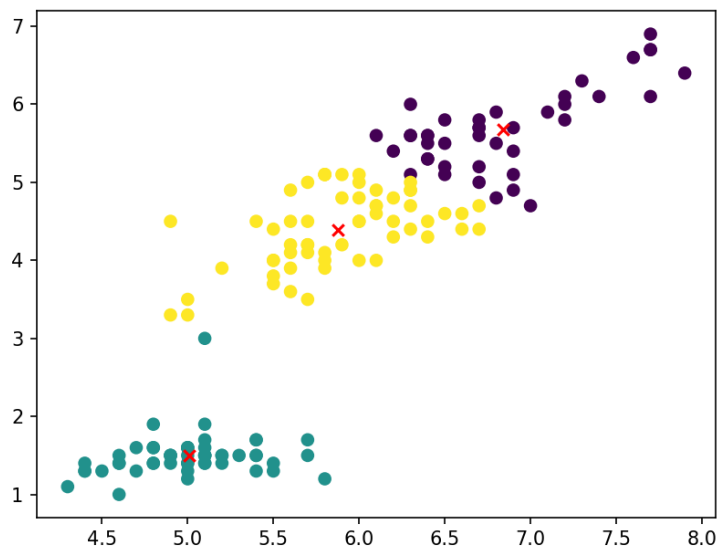


图 27. k-means 算法的聚类情况

2. 数据的实际归类情况:

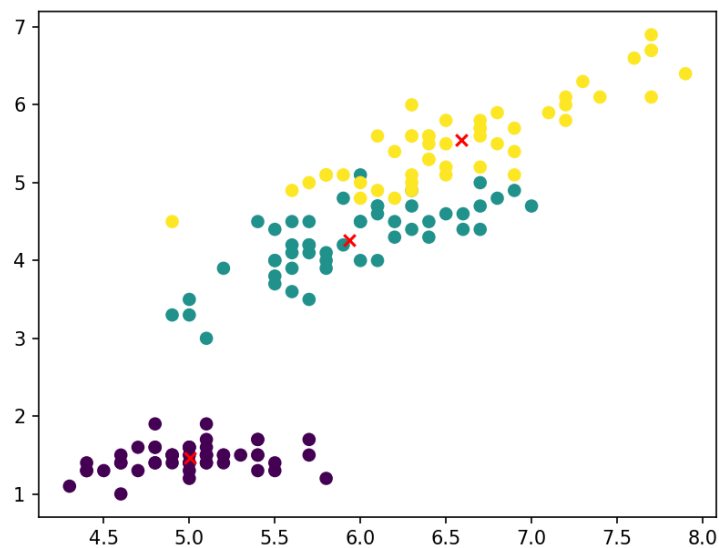


图 28. 实际归类情况

3. 该聚类算法得出准确率为:

准确率为: 0.88

图 29. Kmeans 准确率

混合高斯算法:

1. 混合高斯算法聚类情况:

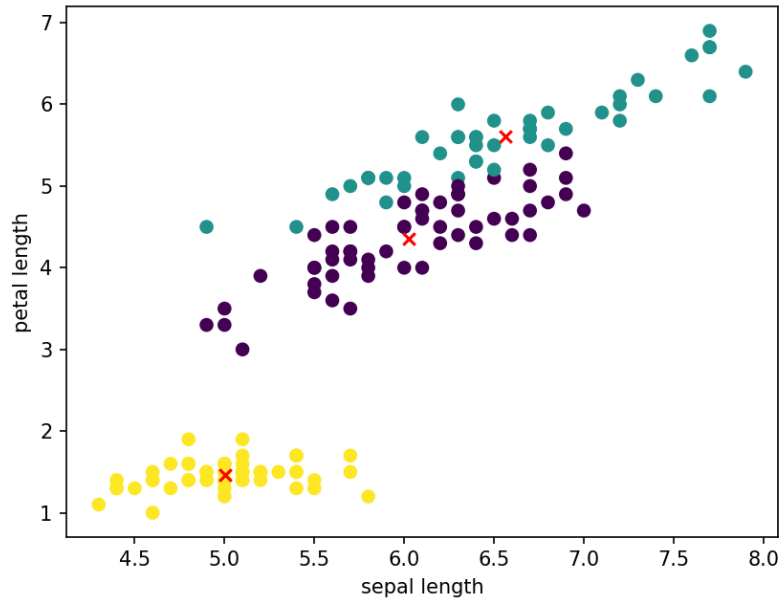


图 30. 混合高斯聚类结果

2. 数据实际分类情况:

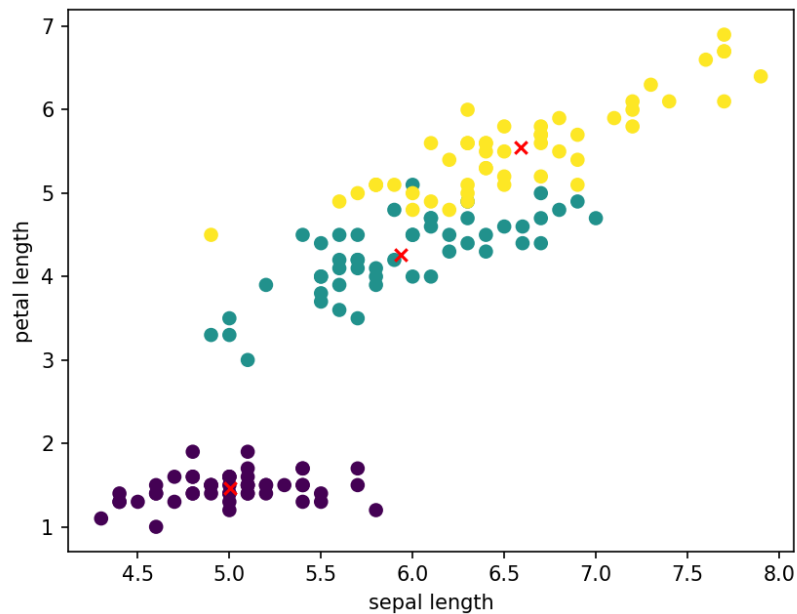


图 31. 实际归类情况

3. 此时该聚类算法得出准确率为:

准确率为: 0.92

图 32. 混合高斯聚类准确率

五、实验体会

通过本次 IRIS 鸢尾花数据集的聚类分析实验，我对无监督学习中的聚类算法有了更深刻的理解。在数据准备与预处理阶段，我了解了 IRIS 数据集的基本结构（150 个样本、4 个特征、3 个类别），并挑选了花萼长度和花瓣长度这两个最具区分度的特征作为聚类依据，体现了特征工程的重要性。在 K-means 算法实现中，我遇到了距离比较逻辑错误的问题，通过调试修正了条件语句，采用了清晰的 if-elif-else 逻辑。这让我深刻

体会到编程严谨性的重要性。经过 100 次迭代，算法成功将样本聚为三类，聚类中心与真实中心较为接近。在混合高斯(GMM-EM)聚类中，我认识到该算法通过 EM 算法计算后验概率，相比 K-means 的确定性分配更加灵活科学。同时我也发现 GMM-EM 对初始参数较为敏感，需要多次尝试以获得稳定结果。在算法比较方面，K-means 实现简单、计算快速，适合大规模数据；GMM-EM 虽然计算复杂度更高，但能提供概率分配信息，适合对聚类质量要求更高的场景。在模型评估阶段，我通过多种标签映射组合计算准确率，认识到聚类评估的复杂性。可视化结果直观展示了两种算法均能有效识别三个鸢尾花品种。本次实验让我掌握了 K-means 和 GMM-EM 的原理与实现，加深了对机器学习完整流程的理解，并培养了我分析解决实际问题的能力。

教师评语：

指导教师：

日 期：